Caching and CDN Design Guide

Halcyon Labs Platform Engineering · Internal standard PE-CACHE-07 · Revision 3

1. Why we cache

Caching is the practice of storing the result of an expensive operation so that later requests can be served faster and more cheaply. At Halcyon Labs almost every read path passes through at least one cache layer before it touches a database. A well-designed cache reduces latency, protects downstream systems from load spikes and lowers infrastructure cost. A badly designed cache serves stale data, hides bugs and fails in surprising ways during incidents.

This guide explains the general principles first and then describes the specific defaults that Halcyon Labs teams must follow. Where this document says *must*, a deviation needs approval from the Platform Architecture Review, which meets every second Thursday.

2. Cache placement and tiers

Caches can live in several places: in the client (browser or mobile app), at the edge (a CDN), in a shared distributed cache such as Redis, or inside the application process itself. Each tier trades freshness for speed. The closer a cache is to the user, the faster it is, and the harder it is to invalidate.

Halcyon Labs uses four tiers. The table below lists the standard configuration for each one. Teams may lower a TTL without approval, but raising it above the listed value requires review.

Tier	Technology	Typical latency	Default TTL	Size limit
T0 – Client	HTTP Cache-Control headers	0 ms (local)	30 seconds	Browser managed
T1 – Edge	CDN (provider: Stratus Edge)	8–20 ms	10 minutes for static, 0 for HTML	Unlimited (billed per GB)
T2 – Shared	Redis cluster "Kestrel"	0.6–1.2 ms	47 seconds	96 GB per cluster
T3 – In-process	Caffeine (JVM) / lru-cache (Node)	< 0.05 ms	5 seconds	2,048 entries per instance

2.1 Why the shared cache TTL is 47 seconds

The 47-second default for the Kestrel cluster is deliberate. Our pricing service republishes prices every 60 seconds, and product owners agreed that a customer must never see a price older than one full publishing cycle. A 47-second TTL plus up to 13 seconds of propagation delay keeps the worst-case staleness under 60 seconds. We also avoided round numbers such as 30 or 60 seconds because many keys created at the same moment with a round TTL tend to expire together and cause synchronized load on the database. In addition, every TTL in Kestrel gets a random jitter of plus or minus 10 percent.

3. Caching patterns

3.1 Cache-aside (lazy loading)

The application first checks the cache. On a miss it reads from the database, writes the result into the cache and returns it. Cache-aside is the default pattern at Halcyon Labs because the cache only ever contains data that somebody asked for, and a cache outage degrades performance instead of breaking correctness.

3.2 Write-through and write-behind

- **Write-through:** every write goes to the cache and the database synchronously. Reads are always fresh, but writes are slower. Halcyon uses write-through only for the user session store.
- **Write-behind (write-back):** writes go to the cache and are flushed to the database later in batches. This is fast but risks data loss if the cache node dies. Write-behind is **banned** at Halcyon Labs for any data involving money, after the incident described in section 6.
- **Refresh-ahead:** the cache proactively reloads hot keys before they expire. We use it for the top 500 product pages only.

4. Invalidation and the stampede problem

Phil Karlton's famous remark that cache invalidation is one of the two hard problems in computer science is quoted often because it is true. The main strategies are time-based expiry (TTL), explicit deletion on write, and versioned keys. Halcyon prefers explicit deletion combined with a TTL as a safety net. Updating the cache value directly on write is discouraged because two concurrent writers can leave the cache holding the older value.

A **cache stampede** (also called the thundering herd or dog-piling) happens when a popular key expires and hundreds of requests miss at the same time, all hitting the database together. Halcyon's standard defence is **probabilistic early expiration** using the XFetch algorithm with a beta value of **1.3**. Each request computes a small random chance of refreshing the key before it actually expires; the chance rises as expiry approaches, so normally exactly one request refreshes it. As a second layer, a request that misses must acquire a short Redis lock (lock TTL 3 seconds) before rebuilding the value; other requests wait up to 150 milliseconds and then serve the stale copy.

5. Content Delivery Networks

A CDN is a geographically distributed set of servers, called points of presence (POPs), that cache content close to users. When a user in Chennai requests an image, the CDN serves it from a nearby POP rather than from the origin server in Frankfurt. CDNs reduce latency, absorb traffic spikes and provide a first line of defence against DDoS attacks.

5.1 Push versus pull CDNs

With a **pull CDN**, the edge fetches content from the origin the first time it is requested and caches it. With a **push CDN**, the team uploads content to the CDN ahead of time. Pull is simpler and is the right choice for most websites. Push makes sense for large files that rarely change, such as software installers or video.

5.2 Halcyon's CDN setup

- Provider: **Stratus Edge**, chosen in 2024 over two competitors mainly for its 38 POPs in South and South-East Asia.

- Target edge cache hit ratio: **92 percent** for static assets. An alert fires if it stays below 85 percent for 15 minutes.
- Static assets use content-hashed filenames (for example *app.3f9a1c.js*) so they can be cached for a year and never need purging.
- HTML responses use *stale-while-revalidate=120*, meaning the edge may serve a page up to 120 seconds old while it fetches a fresh copy in the background.
- Full-site purges are rate-limited to **3 per hour** to protect the origin.

6. Lessons from incidents

The Diwali wallet incident (INC-2291). In November 2024 the wallet team used write-behind caching for balance updates to handle festival traffic. A Kestrel primary node failed before flushing, and 1,140 wallet top-ups were lost from the database for 40 minutes until they were reconstructed from payment-gateway logs. This incident is the reason write-behind is banned for financial data.

The cold-start incident (INC-2417). After a full Kestrel cluster restart in February 2025, the cache was empty and database CPU reached 100 percent within 90 seconds. Since then every Kestrel restart must be preceded by a cache warm-up job that preloads the 50,000 most-read keys.

7. Checklist for new services

1 Use cache-aside unless you have a documented reason not to.

2 Keep the Kestrel TTL at or below 47 seconds and keep the 10 percent jitter.

3 Protect every key that receives more than 200 reads per second with XFetch (beta 1.3).

4 Never cache personally identifiable information at the CDN tier.

5 Expose cache hit ratio, eviction count and p99 latency on your service dashboard.

6 Write a runbook entry for what happens when the cache is completely unavailable.

Questions about this standard go to the #platform-caching channel. The document owner is Arjun Venkatesh, Staff Engineer, Platform Engineering.